Lab-05 Report: Generated 2026-10-27 19:10

Names:

Group Name:

- **Q1.1):** Based on this relationship, what combinations of torque and rotational speed would you expect to be associated with power failures? Would you expect failures to occur when power is too low, too high, or both?
- **A1.1):** If the power is too low, I would expect that the rotational speed or torque would be impaired (they are porportional, so either falling would reduce power). Similarly for too high either or both torque or rotational speed would be too high.

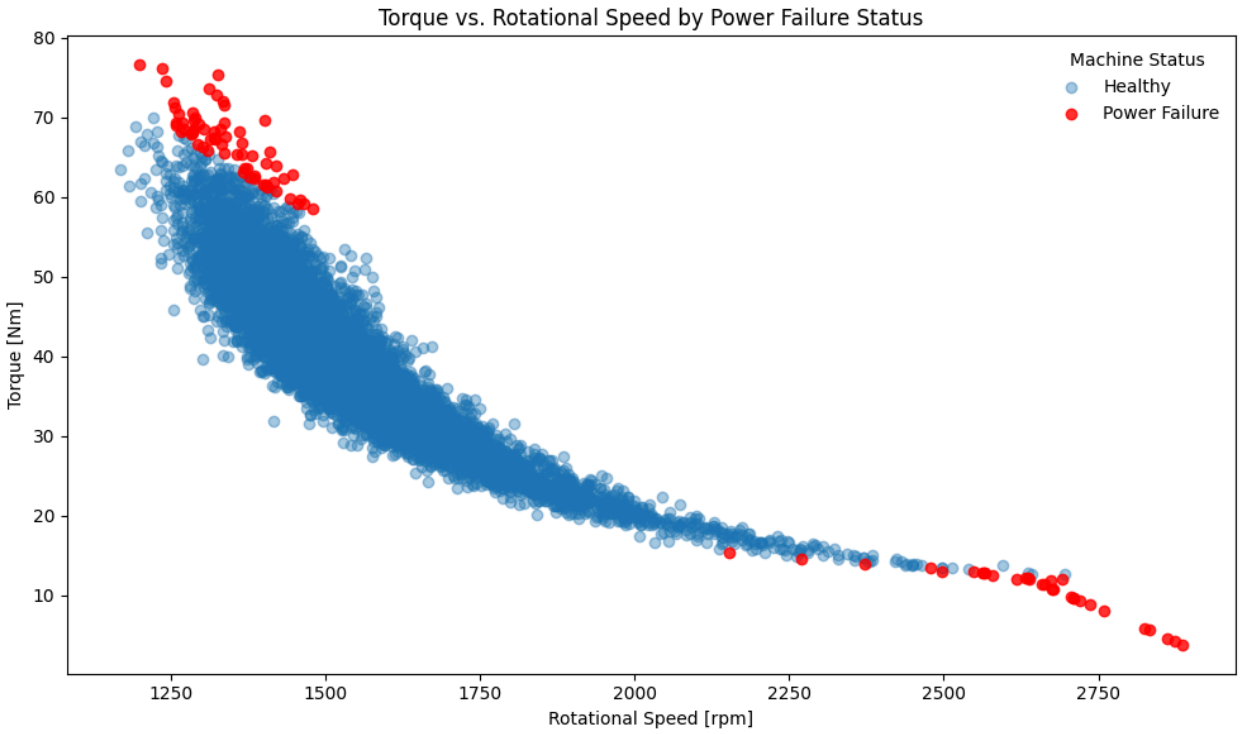
```
# Create the figure matching your dimensions
plt.figure(figsize=(10, 6))

# Separate your healthy points and power failures by selecting on `PWF` column

# YOUR CODE STARTS HERE
healthy = maintenance_df[maintenance_df['PWF'] == 0]
power_failures = maintenance_df[maintenance_df['PWF'] == 1]
plt.scatter(healthy['Rotational speed [rpm]'], healthy['Torque [Nm]'],
            alpha=0.4, label='Healthy')
plt.scatter(power_failures['Rotational speed [rpm]'], power_failures['Torque
[Nm]'],
            color='red', alpha=0.8, label='Power Failure')
plt.xlabel('Rotational Speed [rpm]')
plt.ylabel('Torque [Nm]')
plt.title('Torque vs. Rotational Speed by Power Failure Status')
# YOUR CODE ENDS HERE

plt.legend(title='Machine Status', frameon=True, facecolor='white',
edgecolor='none')

plt.tight_layout()
plt.show()
```




```
# Define the 5 specific engineering failure columns
failure_modes = ['TWF', 'HDF', 'PWF', 'OSF', 'RNF']


# YOUR CODE STARTS HERE
failure_counts = maintenance_df[failure_modes].sum()
print(failure_counts)
# YOUR CODE ENDS HERE

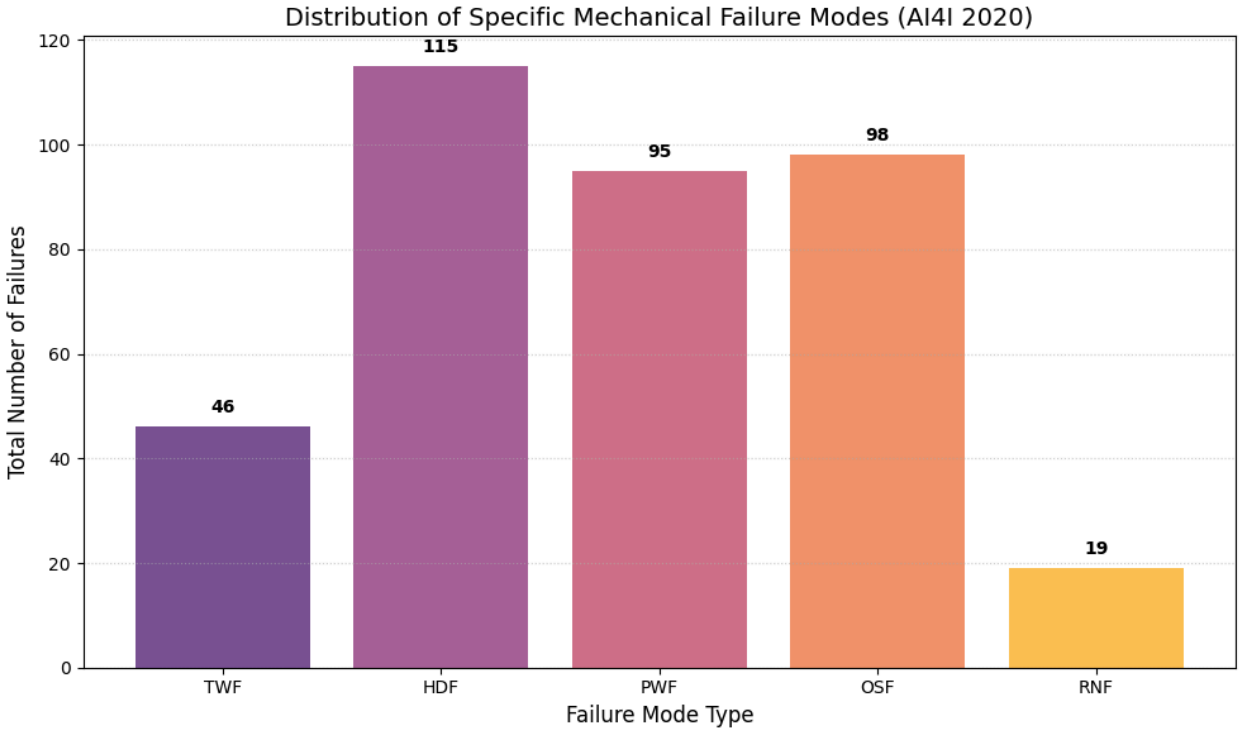
# Create the bar chart
plt.figure(figsize=(10, 6))
plt.bar(failure_counts.index, failure_counts.values,
color=plt.cm.inferno(np.linspace(0.2, 0.8, len(failure_counts.values))),
alpha=0.7)

# Add values on top of each bar for explicit clarity
for i, val in enumerate(failure_counts.values):
    plt.text(i, val + 2, str(val), ha='center', va='bottom',
fontweight='bold')

plt.title('Distribution of Specific Mechanical Failure Modes (AI4I 2020)',
fontsize=14)
plt.xlabel('Failure Mode Type', fontsize=12)
plt.ylabel('Total Number of Failures', fontsize=12)
plt.grid(axis='y', linestyle=':', alpha=0.6)

plt.tight_layout()
plt.show()
```

```
TWF     46
HDF    115
PWF     95
OSF     98
RNF     19
dtype: int64
```



- **Q1.3:** Complete the `__init__` function below, and explain your implementation of the sliding window of length `seq_len`. For this, `X_seq` and `y_multi` lists are provided and will be filled in through your `for` loop implementation using the `.append` function. The tricky part is ensuring your `for` loop condition is set up properly; for this, consider how to ensure you loop through all possible blocks of `seq_len` without running out of room.

- **A1.3:** In order to get optimal validation accuracy I will insure that we have data leakage between the training and validation data (shared sliding window) as described in lecture.

```
class MaintenanceSequenceDataset(d2l.DataModule):
    def __init__(self, df, seq_len=10, batch_size=64):
        super().__init__()
        self.save_hyperparameters()

        # Select continuous engineering features
        feature_cols = ['Air temperature [K]', 'Process temperature [K]',
'Rotational speed [rpm]', 'Torque [Nm]', 'Tool wear [min]']
```

```
        # Select the 5 multi-label failure targets
        target_cols = ['TWF', 'HDF', 'PWF', 'OSF', 'RNF']

        # Main tasks to complete for students:
        # 1. Grab the engineering features and failure targets from the df
DataFrame
        # 2. Standardize the features for numerical stability
        # 3. Build sliding window history sequences. The sequences should be
of length `seq_len` and the target should be the failure state at the final
step of the window.

        X_seq, y_multi = [], []

        # YOUR CODE STARTS HERE
        X = df[feature_cols].values
        y = df[target_cols].values

        scalar = StandardScaler()
        X = scalar.fit_transform(X)

        for i in range(len(X) - seq_len + 1):
            X_seq.append(X[i:i+seq_len])
            y_multi.append(y[i + seq_len - 1])

        self.X = torch.tensor(np.array(X_seq), dtype=torch.float32)
        self.y = torch.tensor(np.array(y_multi), dtype=torch.float32)

        # YOUR CODE ENDS HERE

        self.num_train = int(len(self.X) * 0.8)

    def get_dataloader(self, train):
        i = slice(0, self.num_train) if train else slice(self.num_train, None)
        return self.get_tensorloader((self.X, self.y), train, i)
```

- **Q1.4:** A correct implementation of `MaintenanceSequenceDataset` should generate tensors of size `([batch_size, sequence_length, num_features])` and `([batch_size, num_targets])`. How does this compare to when we previously were exploring tabular data for classification? Briefly explain.
- **A1.4:** Looks like we got it right after selecting self.y correctly. This is different as our data is over a range of (kind of) continuous values rather than just values at one moment in time.

- **Q2.1:** Complete the `__init__` implementation in the code block below. Explain how you made use of parameter `num_hiddens`.

- **A2.1:** num_hiddens is the size of our hidden state, so it is used to determine the size of the output of our hidden calculation. This is all the weight and bias parameters that output to the hidden state: Wxh, Whh, bh. This is also used for the final output weight: Whq.

```
class RNNClassifier_v1(d2l.Classifier):
    def __init__(self, input_size=5, num_hiddens=64, num_outputs=5, lr=0.01,
pos_weight=None):
        super().__init__()
        self.save_hyperparameters()

        # Manually initialize parameters


# YOUR CODE STARTS HERE
        self.W_xh = torch.nn.Parameter(torch.normal(0, 0.01, (input_size,
num_hiddens), dtype=torch.float32))
        self.W_hh = torch.nn.Parameter(torch.normal(0, 0.01, (num_hiddens,
num_hiddens), dtype=torch.float32))
        self.W_hy = torch.nn.Parameter(torch.normal(0, 0.01, (num_hiddens,
num_outputs), dtype=torch.float32))
        self.b_h = torch.nn.Parameter(torch.zeros(num_hiddens,
dtype=torch.float32))
        self.b_y = torch.nn.Parameter(torch.zeros(num_outputs,
dtype=torch.float32))

# YOUR CODE ENDS HERE

        # Store a parameter that represents an increase in the loss associated
with false negatives
        self.pos_weight = pos_weight
```

- **Q2.2:** Complete the `forward` implementation in the code block below. You will need to initialize the hidden state `H`, process the input sequence one time-step `t` at a time by looping through `seq_len`, and at each time-step, update the hidden state using the recurrent update equation provided above.

  Explain how in your code your hidden state `H` is initialized, and where it is updated for the time steps in your sequence.

- **A2.2:** Our hidden state is initialized as all zeros if it does not exist, then for the number of sequences we do the equation for the `H + 1`:

$$\mathbf{H}_t = \phi(\mathbf{X}_t \mathbf{W}_{\text{xh}} + \mathbf{H}_{t-1} \mathbf{W}_{\text{hh}} + \mathbf{b}_{\text{h}})$$

```
@d2l.add_to_class(RNNClassifier_v1)
def forward(self, X, H=None):
```

```
        batch_size, seq_len, _ = X.shape

        # Initialize the hidden state to zeros if not provided

# YOUR CODE STARTS HERE
        if H is None:
                H = torch.zeros(batch_size, self.num_hiddens, device=X.device)

        for i in range(seq_len):
                H = torch.tanh(torch.matmul(X[:,i,:], self.W_xh) +
torch.matmul(H, self.W_hh) + self.b_h)


# YOUR CODE ENDS HERE

        # Use the updated hidden state to compute the output logits for multi-
label classification
        return torch.matmul(H, self.W_hy) + self.b_y
```

- **Q2.3:** To understand the effect of `pos_weight`, simplify the loss function for the two possible label values: $y = 0$ and $y = 1$. How does $\mathbf{p}$, corresponding to the `pos_weight` parameter, affect the loss in both cases? Why is this useful for highly imbalanced datasets such as the predictive maintenance dataset?
- **A2.3:** because we only ahve two classes, pos_weight is effectively a penalty (or potentially a positive scalar) applied to one of the classes to account for the larger embalance.

```
maintenance_df = pd.read_csv('data/ai4i2020.csv')
MaintenanceData = MaintenanceSequenceDataset(df=maintenance_df, seq_len=10,
batch_size=64)

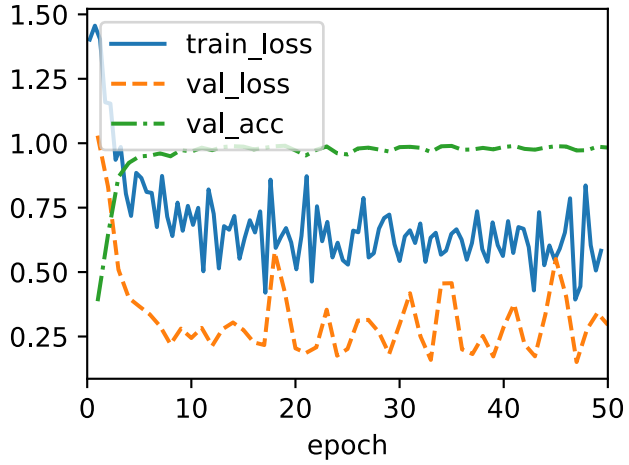
# YOUR CODE STARTS HERE
print(f"Maintentence: {maintenance_df.shape}")
targets = maintenance_df[failure_modes]
negatives = (targets == 0).sum(axis=0)
positives = (targets == 1).sum(axis=0)
calculated_weights = torch.tensor((negatives / (positives + 1)).values,
dtype=torch.float32)
# YOUR CODE ENDS HERE


model_1 = RNNClassifier_v1(input_size=5, num_hiddens=64, num_outputs=5,
lr=0.05, pos_weight=calculated_weights)

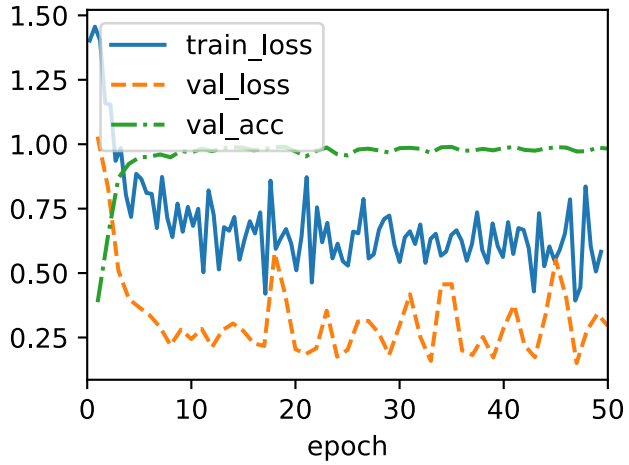
# Fit pipeline
trainer_1 = d2l.Trainer(max_epochs=50, gradient_clip_val=1.0)
trainer_1.fit(model_1, MaintenanceData)
```

```
#Print results
print_model_results(model_1.board)
```

```
Maintentence: (10000, 14)
```




```
========================================
Final Training Loss:     0.5827
Final Validation Loss:   0.2952
----------------------------------------
Final Validation Accuracy: 98.31%
========================================
```



- **Q2.4:** Recall that `pos_weight` was introduced to help counter class imbalance by increasing the penalty for missed failures. Based on the confusion matrices, was this strategy successful? Which failure modes exhibited the highest false negative rates, and what might explain their difficulty?
- **A2.4:** It looks like it it did at least a little. We got like, basically 98% accuracy which is a bit better than always guessing normal. The highst false negative appears to be TWF at 81 Predicted fails but it actually being good. TWF makes sense at being harder to predict, as its just breaking with age rather than having

a much more obvious catestrophic failure and the model might just be predicting the failure before it actually fails.

- **Q2.5:** Complete the code block below to train our concise implementation (`RNNClassifier_v2`) using the same hyperparameters as before. How does the accuracy and computational performance compare to our from-scratch version?
- **A2.5:** Our accuracy is the same, at 98.3% (I dont know if this is a mistake?) but the cell did take about 1 second less to compute so maybe it is more efficent?

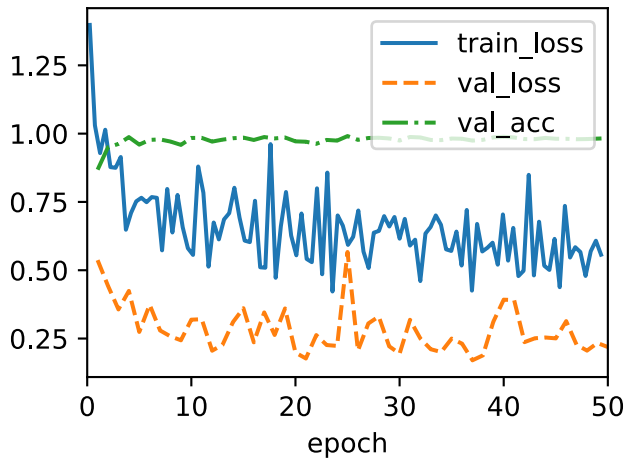
```
# Instantiate the model with the same hyperparameters as before, but now using
the concise RNN implementation

# YOUR CODE STARTS HERE
model_2 = RNNClassifier_v2(input_size=5, num_hiddens=64, num_outputs=5,
lr=0.05, pos_weight=calculated_weights)

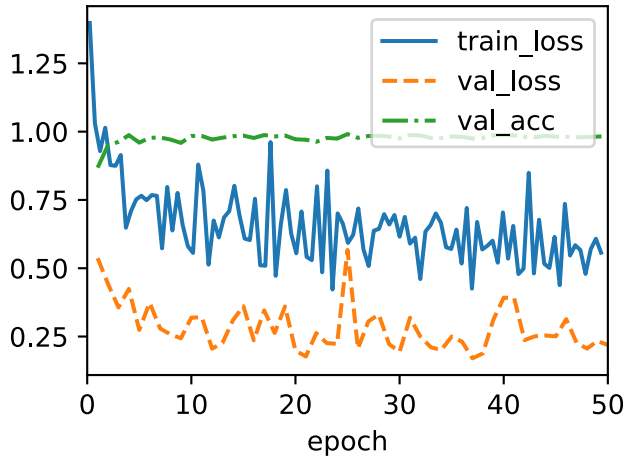
# Fit pipeline
trainer_2 = d2l.Trainer(max_epochs=50, gradient_clip_val=1.0)
trainer_2.fit(model_2, MaintenanceData)

#Print results
print_model_results(model_2.board)
# YOUR CODE ENDS HERE
```

```
/tmp/ipykernel_3033536/3879190811.py:16: UserWarning: To copy construct from a
tensor, it is recommended to use sourceTensor.detach().clone() or
sourceTensor.detach().clone().requires_grad_(True), rather than
torch.tensor(sourceTensor).
  self.pos_weight = torch.tensor(pos_weight, dtype=torch.float32)
```



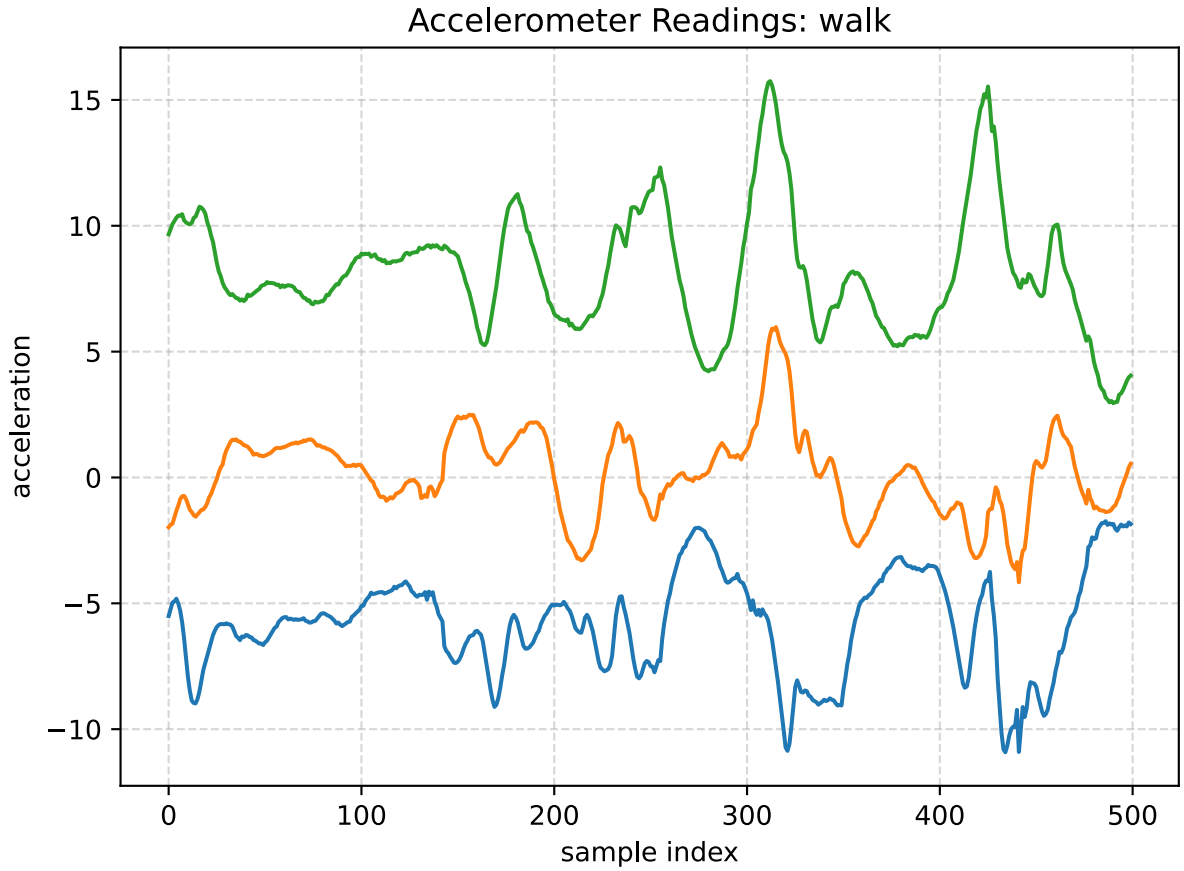
```
========================================
Final Training Loss:     0.5569
Final Validation Loss:   0.2189
----------------------------------------
Final Validation Accuracy: 98.32%
========================================
```

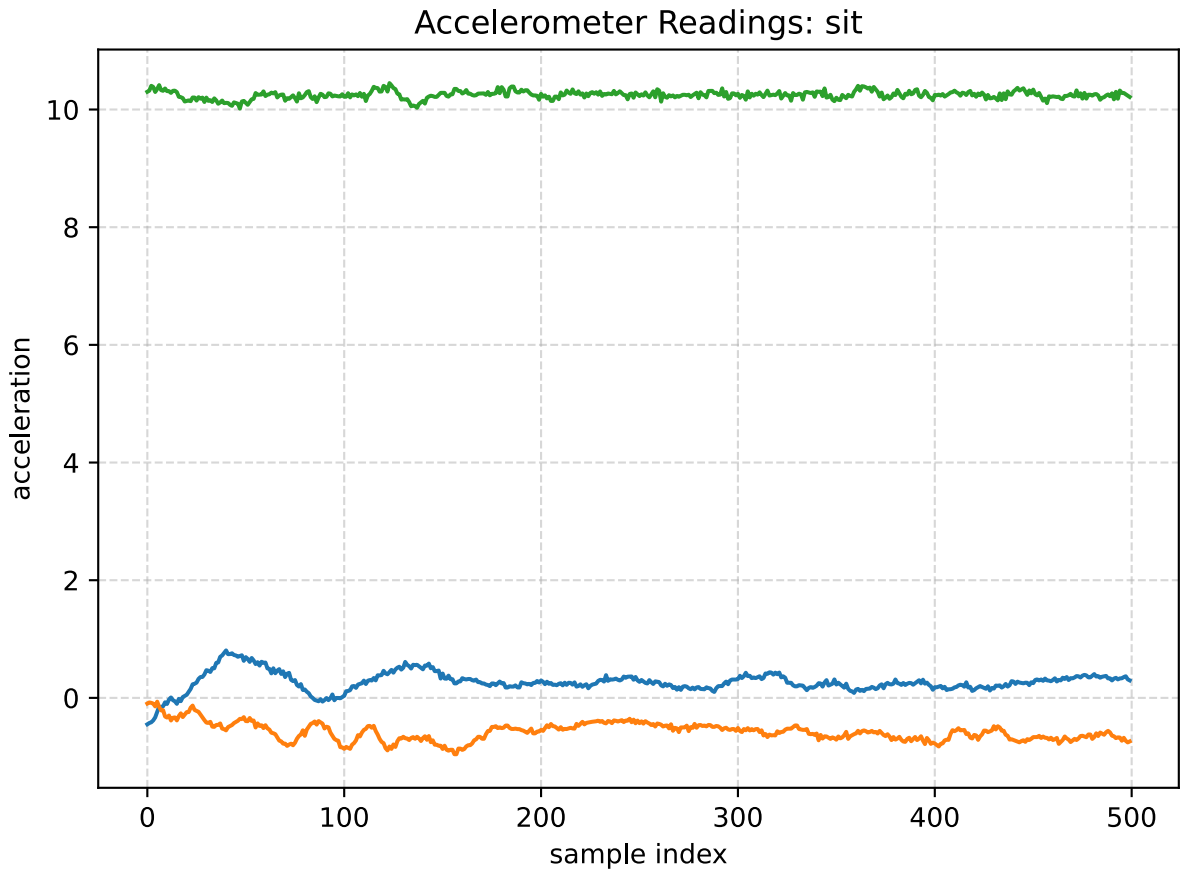


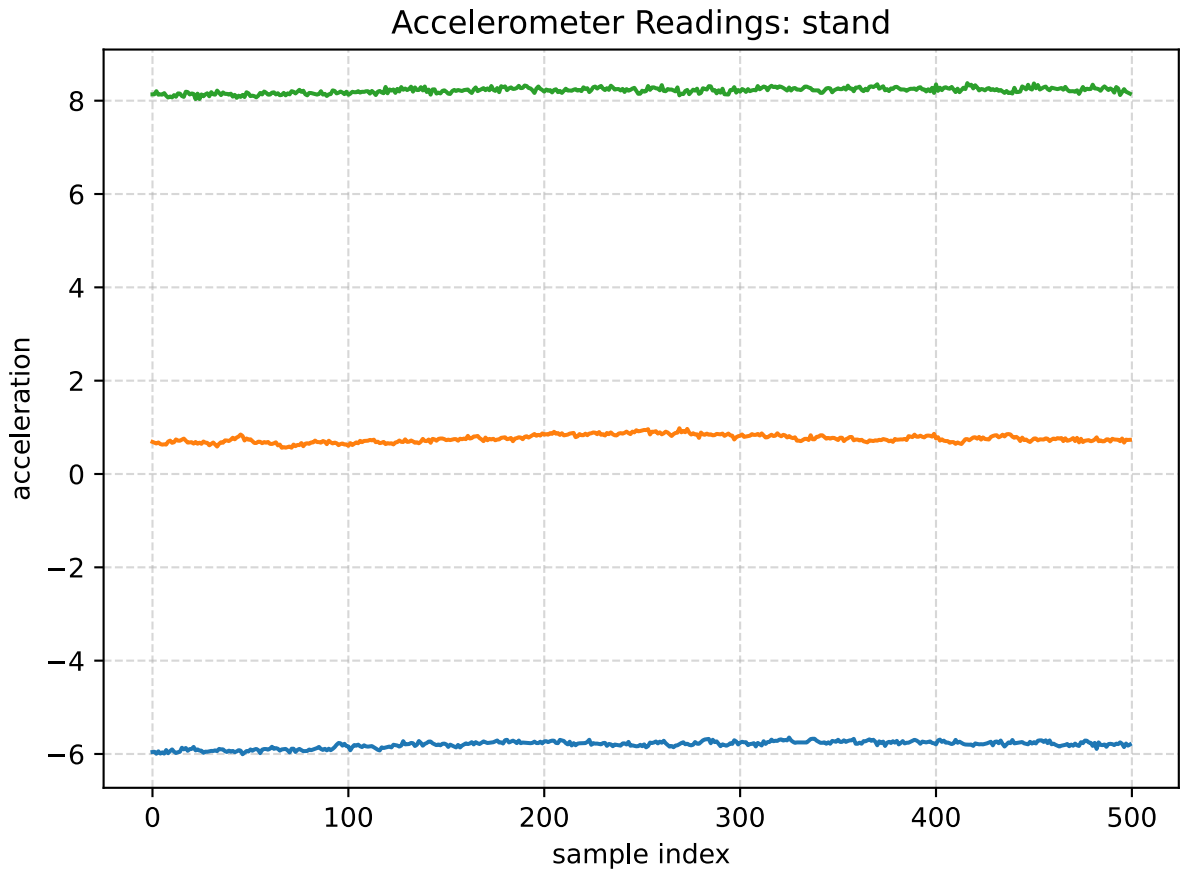

```python
def plot_activity(df, activity_name, num_samples=500):
    # Filter for the target activity
    activity_df = df[df['gt'] == activity_name].iloc[:num_samples]

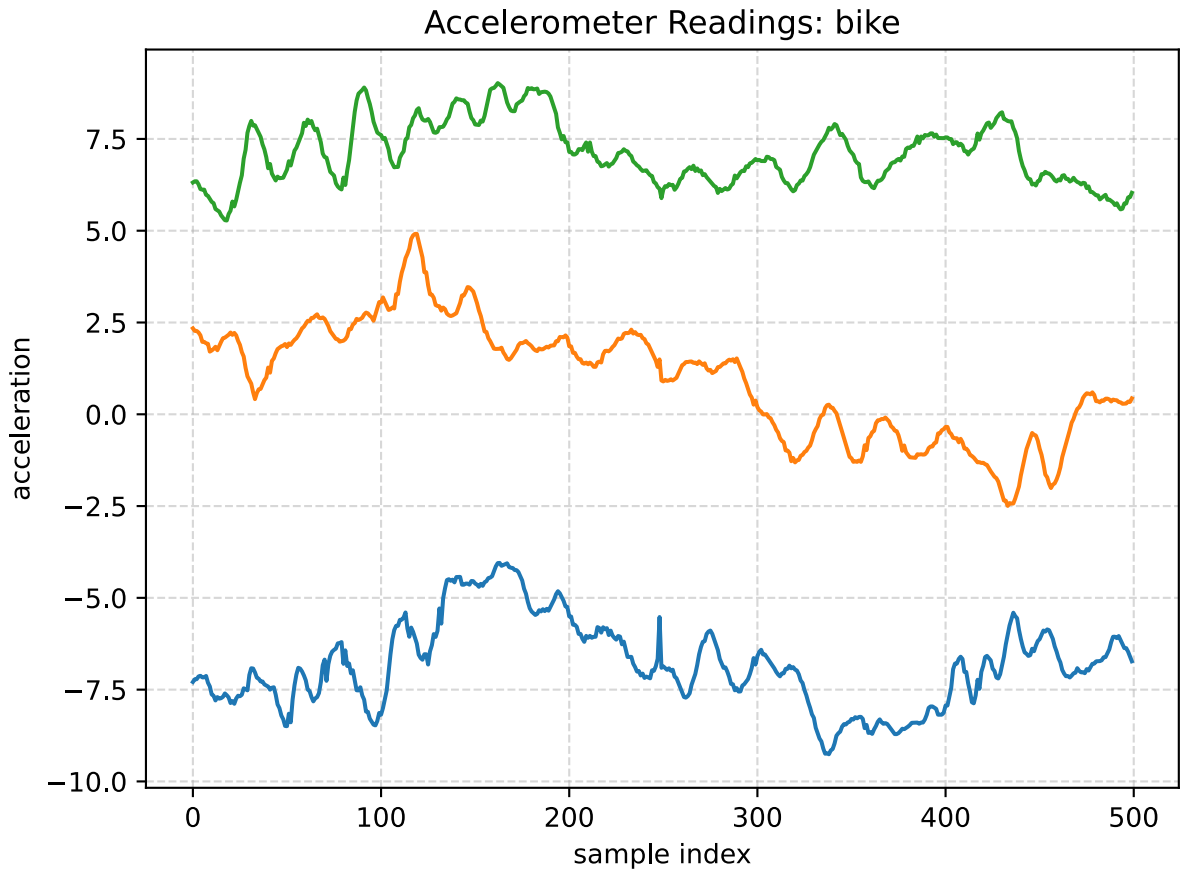
# YOUR CODE STARTS HERE
    #print(f"activity_df shape: {activity_df.shape}")
    plt.plot(activity_df['x'].values, label='x')
    plt.plot(activity_df['y'].values, label='y')
    plt.plot(activity_df['z'].values, label='z')
    plt.title(f"Accelerometer Readings: {activity_name}")
    plt.xlabel("sample index")
    plt.ylabel("acceleration")
    plt.grid(True, linestyle='--', alpha=0.5)
    plt.tight_layout()
    plt.show()
# YOUR CODE ENDS HERE

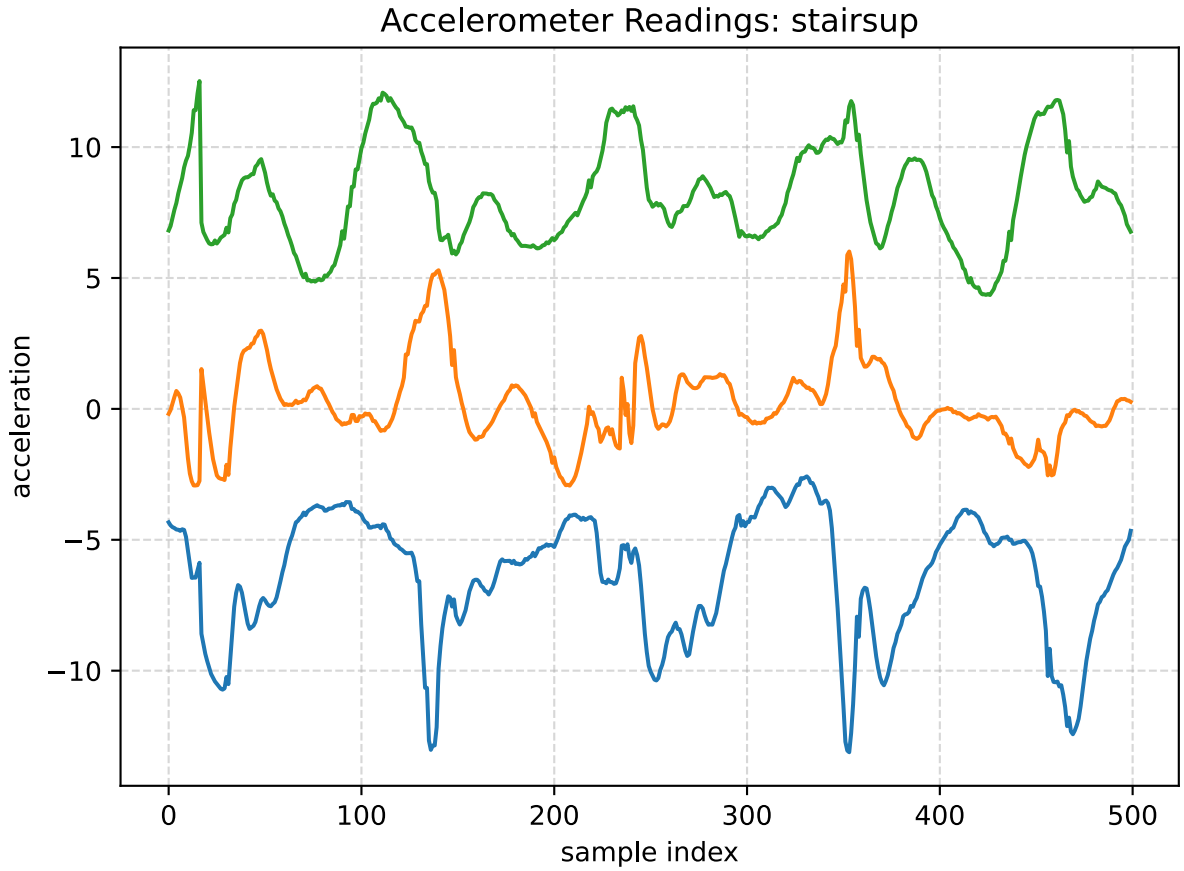
# Run the visualizer for comparison
plot_activity(har_df, 'walk')
plot_activity(har_df, 'sit')
plot_activity(har_df, 'stand')
plot_activity(har_df, 'bike')
plot_activity(har_df, 'stairsup')
plot_activity(har_df, 'stairsdown')
```

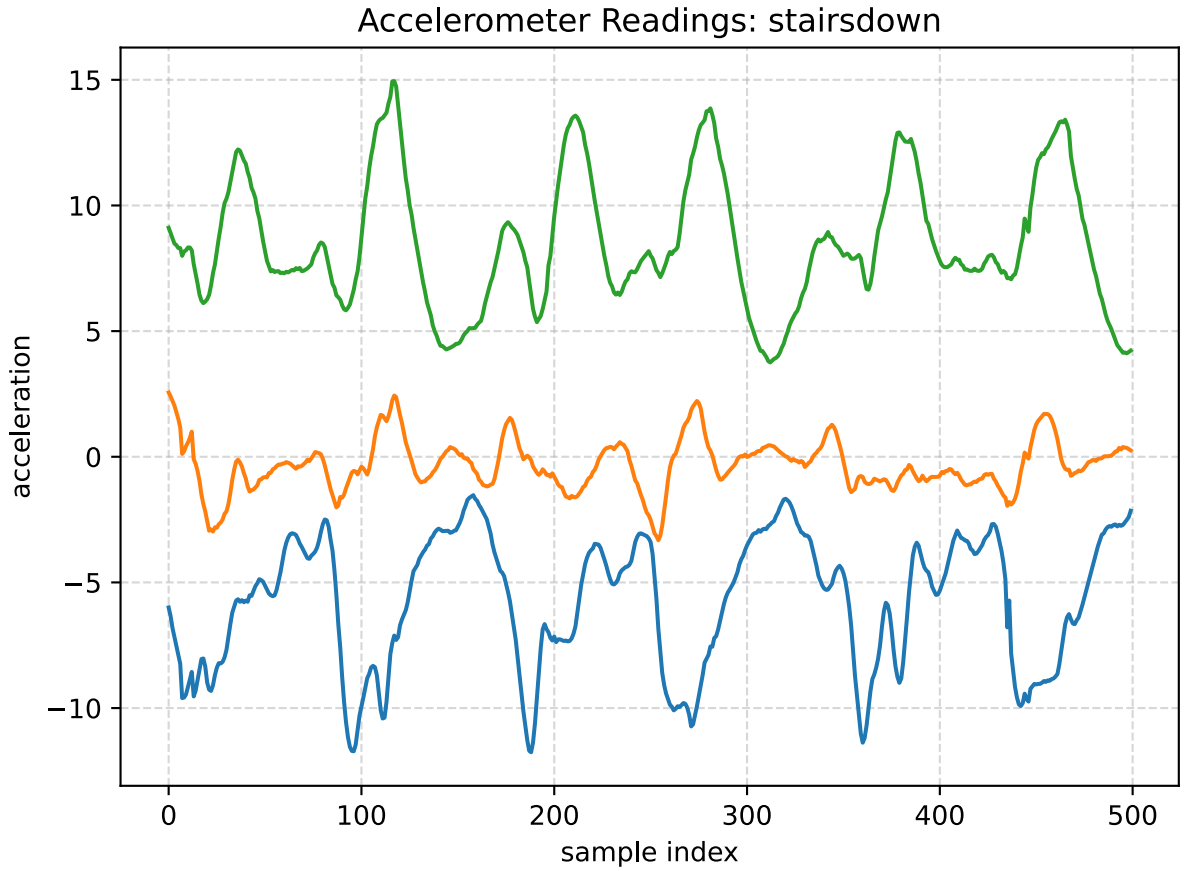

Accelerometer Readings: walk
acceleration
sample index
15
10
5
0
−5
−10
0
100
200
300
400
500


Accelerometer Readings: sit
acceleration
sample index
10
8
6
4
2
0
0
100
200
300
400
500


Accelerometer Readings: stand
acceleration
sample index
8
6
4
2
0
−2
−4
−6
0
100
200
300
400
500


Accelerometer Readings: bike
acceleration
sample index
7.5
5.0
2.5
0.0
−2.5
−5.0
−7.5
−10.0
0
100
200
300
400
500


Accelerometer Readings: stairsup
acceleration
sample index
10
5
0
−5
−10
0
100
200
300
400
500


Accelerometer Readings: stairsdown
acceleration
sample index
15
10
5
0
−5
−10
0
100
200
300
400
500

- **Q3.1:** Accelerometer readings provide linear acceleration ($a(t) = \frac{d^2x}{dt^2}$, or the second derivative of position with respect to time). At first glance, it may seem that recovering a person's path would be straightforward: if acceleration is the second derivative of position, then position can be obtained by integrating twice ($x(t) = \iint a(t)\, dt$). Why would reconstructing a user's path with pure integration be difficult here? How does an RNN solve this problem?
- **A3.1:** Well, not only would be taking two integrals be computationally expensive depending on how fine-grained you want the data, you technically dont know the starting velocity or position (which to be fair an RNN would likely also not solve this). Because an RNN is, well, recurrent by nature it would remember the previous input features without needing to constantly do full integration calculations (hence, much less computationally demanding).

- **Q3.2:** Examine the accelerometer signals for the sitting and standing activities. Although the individual is largely stationary, the sensor readings are not all zero. What physical phenomenon produces this persistent baseline acceleration? Why does this acceleration appear on one or more of the sensor axes even when the subject is not moving?
- **Q3.2:**

```
def create_windows(features, labels, seq_len=50, stride=25):
    # Segment streams into temporal sequence windows
    X_windows = []
    y_windows = []

# YOUR CODE STARTS HERE
    for i in range(0, len(features) - seq_len + 1, stride):
        window = features[i : i + seq_len]
        target = labels[i + seq_len - 1]

        X_windows.append(window)
        y_windows.append(target)

    X_tensor = torch.tensor(np.array(X_windows))
    y_tensor = torch.tensor(np.array(y_windows))

# YOUR CODE ENDS HERE
    return X_tensor, y_tensor
```

- **Q3.3:** Finish the `HARDataset` class in the code block below. You will need to extract and standardize the features, and map the labels to integer values before calling `create_windows()`. Recall from previous labs that `pd.factorize()` can be used to return the integer-coded labels and classes in one tuple. Explain / defend the steps in your implementation.

- **A3.3:**

```
class HARDataset(d2l.DataModule):
    def __init__(self, df, seq_len=200, stride=20, batch_size=64,
max_rows=500000):
        super().__init__()
        self.save_hyperparameters()

        # 1. Filter out unlabeled transitional periods ("null" class)
        df = df[df['gt'] != 'null'].dropna()

        # 2. Extract input dimensions
        # 3. Standardize features
        # 4. Map labels to integer codes

        # YOUR CODE STARTS HERE
        # 2. Extract input dimensions

        if max_rows is not None:
            df = df.iloc[:max_rows]
        # 2. Extract input dimensions (3 accelerometer axes: x, y, z)
        feature_cols = ['x', 'y', 'z']
        features = df[feature_cols].values
        # 3. Standardize features to mean=0, std=1 (converted to float32 for
PyTorch)
        scaler = StandardScaler()
        features = scaler.fit_transform(features).astype(np.float32)
        # 4. Map labels to integer codes and retain the class names in
self.classes_
        labels, self.classes_ = pd.factorize(df['gt'])


        # YOUR CODE ENDS HERE

        self.num_classes = len(self.classes_)

        # Call our helper method
        self.X, self.y = create_windows(features, labels, seq_len, stride)

        # 5. Apply an 80/20 train/validation index separation boundary
        total_samples = self.X.shape[0]
        self.num_train = int(total_samples * 0.8)
        self.num_val = total_samples - self.num_train
```

```
        print(f"Total processed sequence samples: {total_samples}")
        print("X Input Feature Block Shape:", self.X.shape)
        print("y Target Matrix Label Shape:", self.y.shape)

    def get_dataloader(self, train):
        i = slice(0, self.num_train) if train else slice(self.num_train, None)
        return self.get_tensorloader((self.X, self.y), train, i)
```

```
class LSTM_layer(d2l.Module):
    def __init__(self, num_inputs, num_hiddens, sigma=0.01):
        super().__init__()
        self.save_hyperparameters()

        # INPUT GATE PARAMETERS
        self.W_xi = torch.nn.Parameter(torch.randn(num_inputs, num_hiddens) *
sigma)
        self.W_hi = torch.nn.Parameter(torch.randn(num_hiddens, num_hiddens) *
sigma)
        self.b_i  = torch.nn.Parameter(torch.zeros(num_hiddens))

        # FORGET GATE PARAMETERS
        # OUTPUT GATE PARAMETERS
        # CANDIDATE MEMORY PARAMETERS

# YOUR CODE STARTS HERE
        # FORGET GATE PARAMETERS
        self.W_xf = torch.nn.Parameter(torch.randn(num_inputs, num_hiddens) *
sigma)
        self.W_hf = torch.nn.Parameter(torch.randn(num_hiddens, num_hiddens) *
sigma)
        self.b_f  = torch.nn.Parameter(torch.zeros(num_hiddens))
        # OUTPUT GATE PARAMETERS
        self.W_xo = torch.nn.Parameter(torch.randn(num_inputs, num_hiddens) *
sigma)
        self.W_ho = torch.nn.Parameter(torch.randn(num_hiddens, num_hiddens) *
sigma)
        self.b_o  = torch.nn.Parameter(torch.zeros(num_hiddens))
        # CANDIDATE MEMORY PARAMETERS
        self.W_xc = torch.nn.Parameter(torch.randn(num_inputs, num_hiddens) *
sigma)
        self.W_hc = torch.nn.Parameter(torch.randn(num_hiddens, num_hiddens) *
sigma)
        self.b_c  = torch.nn.Parameter(torch.zeros(num_hiddens))
# YOUR CODE ENDS HERE
```

- **Q4.1:** Why do the input, forget, and output gates use a sigmoid activation, while the candidate memory uses a tanh activation?
- **A4.1:**

```python
@d2l.add_to_class(LSTM_layer)
def forward(self, inputs, H_C=None):
    # inputs shape: (batch_size, seq_len, num_inputs) -> e.g., [64, 200, 3]
    batch_size, seq_len, _ = inputs.shape

    # Initialize hidden state (H) and memory cell state (C) if not provided
    if H_C is None:
        H = torch.zeros((batch_size, self.num_hiddens))
        C = torch.zeros((batch_size, self.num_hiddens))
    else:
        H, C = H_C

    outputs = []

    # Loop across sequence
    for t in range(seq_len):

        # Extract the 3D feature slice for time-step t: shape (batch_size,
num_inputs)
        X = inputs[:, t, :]

# YOUR CODE STARTS HERE
        # 1. Input Gate
        I = torch.sigmoid(torch.matmul(X, self.W_xi) + torch.matmul(H,
self.W_hi) + self.b_i)
        # 2. Forget Gate
        F = torch.sigmoid(torch.matmul(X, self.W_xf) + torch.matmul(H,
self.W_hf) + self.b_f)
        # 3. Output Gate
        O = torch.sigmoid(torch.matmul(X, self.W_xo) + torch.matmul(H,
self.W_ho) + self.b_o)
        # 4. Candidate Memory Cell
        C_tilde = torch.tanh(torch.matmul(X, self.W_xc) + torch.matmul(H,
self.W_hc) + self.b_c)
        # 5. Cell State Update
        C = F * C + I * C_tilde
        # 6. Hidden State Update
        H = O * torch.tanh(C)
# YOUR CODE ENDS HERE

        outputs.append(H)

    # Stack the sequence outputs along dim 1: (batch_size, seq_len,
num_hiddens)
    outputs_tensor = torch.stack(outputs, dim=1)

    # Only the hidden state is passed to the next time-step, but we return
both H and C for potential use in the next sequence
    return outputs_tensor, (H, C)
```

- **Q4.2:** Suppose the forget gate always outputs values very close to 0. What would you expect to happen to the cell state and the model's ability to learn long-term dependencies?
- **A4.2:**

- **Q4.3:** Which activities are classified most accurately, and which are the most challenging for the model? What characteristics of these activities might explain the difference?
- **A4.3:**

- **Q4.4:** Explain the line of code, `final_timestep_hidden = output[:, -1, :]`. How does passing this directly into our final linear classifier head successfully produce the class prediction logits?
- **A4.4:**

- **Q4.5:** In our concise implementation above, we make sure to set about `batch_first=true` as a parameter for our LSTM layer. Briefly review the torch.nn.LSTM documentation. Why was this step necessary?
- **A4.4:**

```
# Implement / explore a deeper LSTM architecture with multiple stacked layers
and dropout regularization to improve generalization and performance on the
HAR dataset.

# YOUR CODE STARTS HERE
import time

# 1. Define LSTMClassifier_v3 supporting configurable depth (num_layers) and
dropout
class LSTMClassifier_v3(d2l.Classifier):
    def __init__(self, num_inputs, num_hiddens, num_classes, num_layers=1,
dropout=0.0, lr=0.001):
        super().__init__()
        self.save_hyperparameters()

        # PyTorch nn.LSTM applies dropout between stacked layers when
num_layers > 1.
        # If num_layers == 1, PyTorch expects dropout=0 to prevent
UserWarnings.
```

```
        lstm_dropout = dropout if num_layers > 1 else 0.0
        self.lstm = torch.nn.LSTM(
            input_size=num_inputs,
            hidden_size=num_hiddens,
            num_layers=num_layers,
            batch_first=True,
            dropout=lstm_dropout
        )

        # Fully connected classification head
        self.fc = torch.nn.Linear(num_hiddens, num_classes)

    def forward(self, X):
        # Pass sequence through stacked LSTM: [batch_size, seq_len,
num_inputs]
        output, (H, C) = self.lstm(X)

        # Select the hidden state from the final time-step: [batch_size,
num_hiddens]
        final_timestep_hidden = output[:, -1, :]

        # Project to class logits: [batch_size, num_classes]
        return self.fc(final_timestep_hidden)

    def configure_optimizers(self):
        return torch.optim.Adam(self.parameters(), lr=self.lr)


# 2. Define the two arrays to form a 3x3 grid (9 models)
depth = [1, 2, 3]          # Number of stacked LSTM layers
dropout = [0.0, 0.2, 0.5]  # Dropout regularization probabilities

# Ensure HARData is present in memory
if 'HARData' not in locals():
    har_df = pd.read_csv('data/HAR_accelerometer.csv')
    HARData = HARDataset(df=har_df, seq_len=200, batch_size=64)

# 3. Train and evaluate all 9 models
results = []
print(f"Comparing {len(depth) * len(dropout)} models (depths={depth},
dropouts={dropout})...\n")

for d in depth:
    for drop in dropout:
        print(f"--> Training Model: depth={d}, dropout={drop} ...")
        start_time = time.time()

        model = LSTMClassifier_v3(
            num_inputs=3,
            num_hiddens=64,
            num_classes=HARData.num_classes,
            num_layers=d,
            dropout=drop,
```

```
            lr=0.001
        )
        # Suppress interactive board animation to keep the notebook output
clean
        model.board.display = False

        trainer = d2l.Trainer(
            max_epochs=10,
            num_gpus=1 if torch.cuda.is_available() else 0
        )
        trainer.fit(model, HARData)
        elapsed_time = time.time() - start_time

        # Extract metrics stored in model.board
        tr_loss = float(model.board.data['train_loss'][-1][1])
        val_loss = float(model.board.data['val_loss'][-1][1])
        val_acc = float(model.board.data['val_acc'][-1][1])

        results.append({
            'depth': d,
            'dropout': drop,
            'train_loss': tr_loss,
            'val_loss': val_loss,
            'val_acc': val_acc,
            'time_sec': elapsed_time
        })
        print(f"    Finished in {elapsed_time:.1f}s | Train Loss:
{tr_loss:.4f} | Val Loss: {val_loss:.4f} | Val Acc: {val_acc * 100:.2f}%\n")

# 4. Tabulate and display comparison results
results_df = pd.DataFrame(results)

print("=" * 68)
print("                   9-MODEL COMPARISON SUMMARY")
print("=" * 68)
print(results_df.to_string(index=False, formatters={
    'train_loss': '{:.4f}'.format,
    'val_loss': '{:.4f}'.format,
    'val_acc': lambda x: f"{x * 100:.2f}%",
    'time_sec': '{:.1f}s'.format
}))
print("=" * 68)

# Pivot table: Validation Accuracy (%) across Depth and Dropout
acc_pivot = results_df.pivot(index='depth', columns='dropout',
values='val_acc') * 100
print("\nValidation Accuracy (%) by (Depth x Dropout):")
print(acc_pivot.round(2))

# 5. Plot comparison curves
plt.figure(figsize=(8, 5))
for drop in dropout:
    subset = results_df[results_df['dropout'] == drop]
```

```
    plt.plot(subset['depth'], subset['val_acc'] * 100, marker='o',
linewidth=2, label=f'dropout = {drop}')

plt.title('HAR Classification: Validation Accuracy vs. Depth & Dropout')
plt.xlabel('LSTM Depth (num_layers)')
plt.ylabel('Validation Accuracy (%)')
plt.xticks(depth)
plt.grid(True, linestyle='--', alpha=0.6)
plt.legend()
plt.tight_layout()
plt.show()
# YOUR CODE ENDS HERE
```

```
Comparing 9 models (depths=[1, 2, 3], dropouts=[0.0, 0.2, 0.5])...

--> Training Model: depth=1, dropout=0.0 ...
    Finished in 9.0s | Train Loss: 0.3857 | Val Loss: 0.7061 | Val Acc: 69.91%

--> Training Model: depth=1, dropout=0.2 ...
    Finished in 8.1s | Train Loss: 0.3224 | Val Loss: 0.8721 | Val Acc: 61.42%

--> Training Model: depth=1, dropout=0.5 ...
    Finished in 8.6s | Train Loss: 0.3077 | Val Loss: 0.6326 | Val Acc: 77.79%

--> Training Model: depth=2, dropout=0.0 ...
    Finished in 10.3s | Train Loss: 0.0887 | Val Loss: 0.2268 | Val Acc:
93.22%

--> Training Model: depth=2, dropout=0.2 ...
    Finished in 9.7s | Train Loss: 0.1311 | Val Loss: 0.4043 | Val Acc: 85.86%

--> Training Model: depth=2, dropout=0.5 ...
    Finished in 9.9s | Train Loss: 0.1838 | Val Loss: 0.3681 | Val Acc: 88.63%

--> Training Model: depth=3, dropout=0.0 ...
    Finished in 11.2s | Train Loss: 0.4936 | Val Loss: 1.0609 | Val Acc:
54.32%

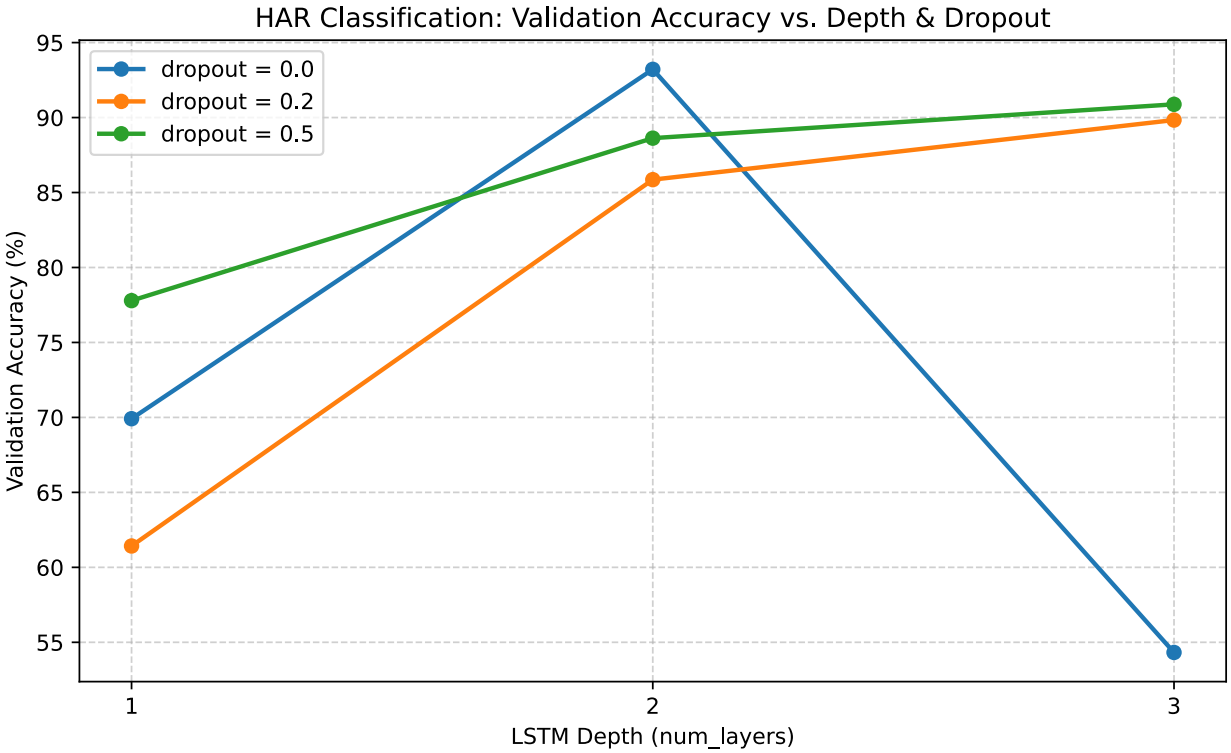
--> Training Model: depth=3, dropout=0.2 ...
    Finished in 12.0s | Train Loss: 0.0988 | Val Loss: 0.3307 | Val Acc:
89.83%

--> Training Model: depth=3, dropout=0.5 ...
    Finished in 12.2s | Train Loss: 0.0804 | Val Loss: 0.3084 | Val Acc:
90.88%

======================================================================
                   9-MODEL COMPARISON SUMMARY
======================================================================
 depth  dropout train_loss val_loss val_acc time_sec
     1      0.0     0.3857   0.7061  69.91%     9.0s
     1      0.2     0.3224   0.8721  61.42%     8.1s
     1      0.5     0.3077   0.6326  77.79%     8.6s
     2      0.0     0.0887   0.2268  93.22%    10.3s
     2      0.2     0.1311   0.4043  85.86%     9.7s
     2      0.5     0.1838   0.3681  88.63%     9.9s
     3      0.0     0.4936   1.0609  54.32%    11.2s
     3      0.2     0.0988   0.3307  89.83%    12.0s
     3      0.5     0.0804   0.3084  90.88%    12.2s
======================================================================

Validation Accuracy (%) by (Depth x Dropout):
dropout    0.0    0.2    0.5
depth
1        69.91  61.42  77.79
```

```
2          93.22  85.86  88.63
3          54.32  89.83  90.88
```



- **Q4.6:** As you varied these settings, how did training time, validation accuracy, and generalization performance change? Does increasing the number of layers consistently improve performance, and does dropout help prevent overfitting?

- **A4.5:** Training time increased as the depth and dropout increased. The depth should be self explanatory as you're doing more and more calculations for each added depth. The dropout adding training time was less obvious but it might be a torch optimization thing? Regardless, the validation accuracy increased going from 1 -> 2 but dropped going from 2 -> 3 with no dropout and very little improvement for layers with dropout. Dropout also appeared to have a sweet spot to have a positive impact on validation accuracy which was not suprising. The most clear example was no dropout which overfit heavily after going from 2-3 layers. I wish we didn't choose to do 9 models though, as total training took forever. Later I needed to rerun the lab because my kernel went stale so I updated the trainer to use GPU and it was much much faster, would reccommend.

The goal of this dataset is to classify different types of cardiac arrhythmias from ECG recordings. If you inspect the dataset, you will notice that the feature columns have generic numeric names and the class labels are encoded as integer values rather

than descriptive categories. The final column contains the target label, with classes indexed from 0 to 4:

- `0` - **N** (**Normal**): A normal heartbeat.
- `1` - **S** (**Supraventricular**): A premature heartbeat originating in the upper chambers of the heart.
- `2` - **V** (**Ventricular**): A premature heartbeat originating in the ventricles. These beats can be associated with more - serious cardiac conditions.
- `3` - **F** (**Fusion**): A hybrid beat formed when a normal beat and a ventricular beat occur simultaneously.
- `4` - **Q** (**Unknown**): A beat that does not clearly fit into the other categories.

To make the dataset easier to work with, replace the scientific-notation feature column names with simple numerical indices and rename the final column to `label`.

```
# Convert the columns to a range of integers for easier handling, and rename
the last column to "label" for clarity

# YOUR CODE STARTS HERE
ecg_df.columns = list(range(ecg_df.shape[1] - 1)) + ["label"]
# YOUR CODE ENDS HERE

print("Last 5 columns after the cleanup:")
print(ecg_df.columns[-5:])
ecg_df.head()
```

```
Last 5 columns after the cleanup:
Index([183, 184, 185, 186, 'label'], dtype='object')
```

	0	1	2	3	4	5	6	7
0	0.960114	0.863248	0.461538	0.196581	0.094017	0.125356	0.099715	0.088319
1	1.000000	0.659459	0.186486	0.070270	0.070270	0.059459	0.056757	0.043243
2	0.925414	0.665746	0.541436	0.276243	0.196133	0.077348	0.071823	0.060773
3	0.967136	1.000000	0.830986	0.586854	0.356808	0.248826	0.145540	0.089202
4	0.927461	1.000000	0.626943	0.193437	0.094991	0.072539	0.043178	0.053541

5 rows × 188 columns

```
plt.figure(figsize=(14, 8))
time_steps = range(187)

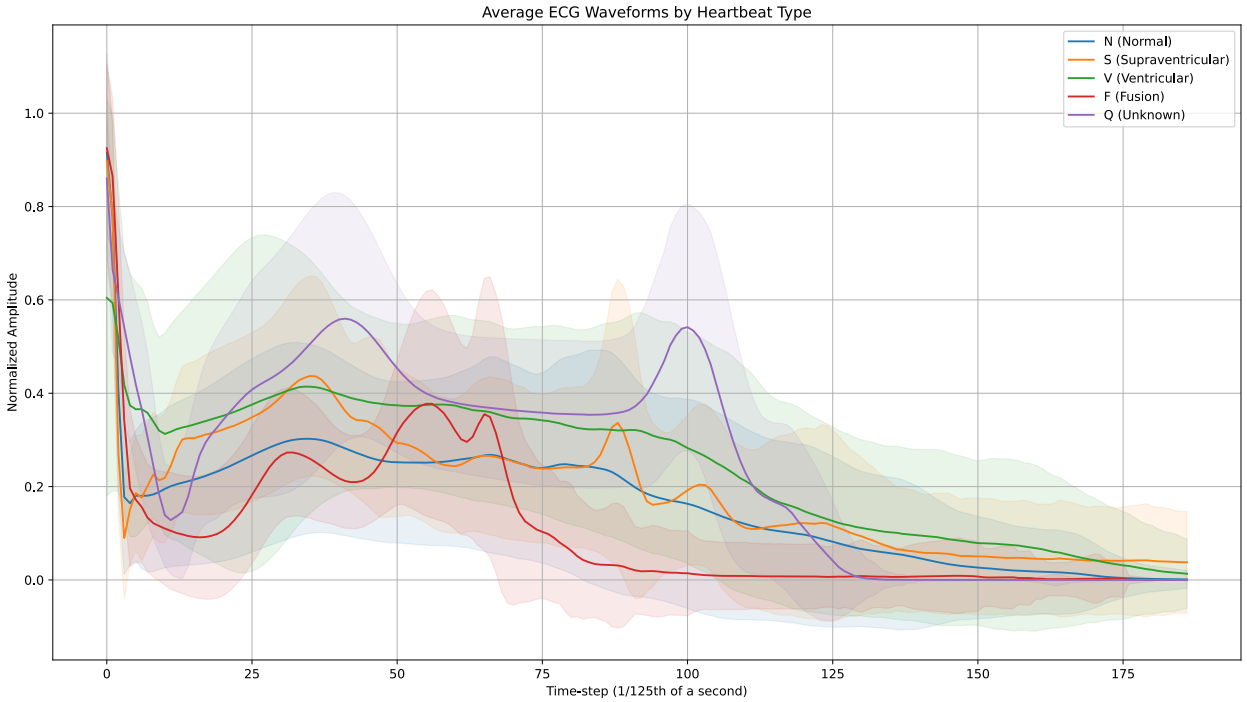
for i, label in enumerate(ecg_labels):

    # Filter for a specific heartbeat type
    # Calculate mean and standard deviation across time steps
```

```
    # Plot the mean waveform with a shaded area representing one standard
deviation above
    # and below the mean (set alpha=0.1 for transparency)

# YOUR CODE STARTS HERE
    heartbeat_data = ecg_df[ecg_df['label'] == i].iloc[:, :-1].values
    # Calculate mean and standard deviation across time steps
    mean_waveform = heartbeat_data.mean(axis=0)
    std_waveform = heartbeat_data.std(axis=0)
    # Plot the mean waveform with a shaded area representing one standard
deviation above
    # and below the mean (set alpha=0.1 for transparency)
    line, = plt.plot(time_steps, mean_waveform, label=label)
    plt.fill_between(
        time_steps,
        mean_waveform - std_waveform,
        mean_waveform + std_waveform,
        color=line.get_color(),
        alpha=0.1
    )
# YOUR CODE ENDS HERE

plt.title("Average ECG Waveforms by Heartbeat Type")
plt.xlabel("Time-step (1/125th of a second)")
plt.ylabel("Normalized Amplitude")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```



- **Q5.2:** The "Unknown" (Q) class exhibits some of the largest and most dramatic waveform peaks in the dataset. Intuitively, it may seem that the most extreme heart activity should be the easiest to diagnose. Why, then, do you think these heartbeats were classified as Unknown rather than assigned to a specific arrhythmia category?
- **A5.2:** It is probably because they are pretty rare heart conditions and would give a model a catch all for "not normal but not one of the other classes". This is safer than a misdiagnoses of normal or an incorrect class.

- **Q6.1:** Suppose our ECG sequence contains a long period of stable behavior followed by a sudden change. Based on the hidden-state update equation, how might the update gate respond during these two phases?
- **A6.1:** It would either choose to take the new data or keep the old data. It would depend on how its trained, but in this case I would expect it to flag and use prioritize the newer input rather than keep the older (as it favors newly computed state if it is closer to 0).

```
class GRU_layer(torch.nn.Module):
    def __init__(self, num_inputs, num_hiddens, sigma=0.01):
        super().__init__()
        self.num_hiddens = num_hiddens

# YOUR CODE STARTS HERE
        # Update gate parameters
        self.W_xz = torch.nn.Parameter(sigma * torch.randn(num_inputs,
num_hiddens))
        self.W_hz = torch.nn.Parameter(sigma * torch.randn(num_hiddens,
num_hiddens))
        self.b_z = torch.nn.Parameter(torch.zeros(num_hiddens))

        # Reset gate parameters
        self.W_xr = torch.nn.Parameter(sigma * torch.randn(num_inputs,
num_hiddens))
        self.W_hr = torch.nn.Parameter(sigma * torch.randn(num_hiddens,
num_hiddens))
        self.b_r = torch.nn.Parameter(torch.zeros(num_hiddens))

        # Candidate hidden state parameters
        self.W_xh = torch.nn.Parameter(sigma * torch.randn(num_inputs,
num_hiddens))
        self.W_hh = torch.nn.Parameter(sigma * torch.randn(num_hiddens,
num_hiddens))
```

```
        self.b_h = torch.nn.Parameter(torch.zeros(num_hiddens))
# YOUR CODE ENDS HERE
```

- **Q6.2:** Finish the `forward()` implementation in the following code block. Below, briefly explain how you handle iterating over a sequence, defining the reset and update gate, and calculate the candidate and final hidden state.
- **A6.2:** We first gather the appropriate X for the current t. Then we basically just follow the equations above, using matrix multiplications to calculate Z and R, then the hidden candidate, then finally the new hidden state.

```
@d2l.add_to_class(GRU_layer)
def forward(self, X):
        batch_size, seq_len, _ = X.shape

        # Initialize the single tracking hidden state (H) to zeros
        H = torch.zeros((batch_size, self.num_hiddens))
        outputs = []

        # The Temporal Core Loop
        for t in range(seq_len):

# YOUR CODE STARTS HERE
                X_t = X[:, t, :]  # Get current timestep of input features

                # Calculate the update and reset gates
                Z = torch.sigmoid(torch.matmul(X_t, self.W_xz) +
torch.matmul(H, self.W_hz) + self.b_z) # Calcualte update gate according to
above equation
                R = torch.sigmoid(torch.matmul(X_t, self.W_xr) +
torch.matmul(H, self.W_hr) + self.b_r) # Calcualte reset gate according to
above equation

                # Calculate the candidate and updated hidden states
                H_tilde = torch.tanh(torch.matmul(X_t, self.W_xh) +
torch.matmul(R * H, self.W_hh) + self.b_h) # hidden candidate
                H = Z * H + (1 - Z) * H_tilde # new hidden state

                # Add a sequence dimension for concatenation
                outputs.append(H.unsqueeze(1)) # unsqueeze adds a dim of size
1 at position 1 (aka the time t above!)
# YOUR CODE ENDS HERE
        outputs_tensor = torch.cat(outputs, dim=1)

        # Return both the full sequence outputs and the final state tuple
        return outputs_tensor, H
```

- **Q6.3:** For our ECG dataset, should class weights be determined solely by how often a class appears in the dataset, or should they also reflect the consequences of making mistakes on that class?
- **A6.3:** They should likely reflect the consequences of making a mistake. A misdiagnosis is much worse than guessing "unknown" as a misdiagnosis could result in incorrect care and death. At least guessing unknown prevents an incorrect treatment as long as a doctor takes it as "the model cannot diagnose" and does diagnostic methods manually.

- **Q6.4:** In the following code block, compute an appropriate set of `class_weights` for use with `CrossEntropyLoss`. You may use inverse frequency weighting, inverse square-root weighting, or a custom scheme tailored to this ECG classification problem. Also, you will want to normalize your class weights such that the average class weight is 1.0. Remember that class weights can reflect not only how rare a heartbeat class is, but also how important it is to correctly identify that class. Defend your choice below and discuss the tradeoffs involved in your weighting strategy.
- **A6.4:** We are going to prioritize classes directly based on the represetnation of the class. This will make the model prioritize the more underrepresented classes. This could unintendedly cause misdiagnosis on the underrepresented class as it is incentivized to make a mistake in the rare direction as the weights are biased in that direction. So something borderline normal might get diagnosed as fusion (the most biased class) by mistake.

```
# Class labels from before
ecg_labels = ['Normal (N)', 'Supraventricular (S)', 'Ventricular (V)', 'Fusion
(F)', 'Unknown (Q)']

# We may or may not have labeled the dataframe, so just grab the 187th column
class_counts = ecg_df.iloc[:, 187].value_counts().sort_index()

# Convert to tensor for PyTorch compatibility
class_counts_tensor = torch.tensor(class_counts.values, dtype=torch.float32)

# YOUR CODE STARTS HERE
class_weights = class_counts_tensor.sum() / (len(class_counts_tensor) *
class_counts_tensor) # Give weight to rare heartbeat classes to prioritize
underrepresented classes.
# YOUR CODE ENDS HERE
```

```
print("Class weights: ")
print(class_weights)
```

```
Class weights:
tensor([ 0.2416,  7.8770,  3.0253, 27.3176,  2.7228])
```

- **Q6.5:** Not all classification errors have the same consequences. Using the confusion matrix, identify the most concerning mistakes made by the model. Which heartbeat classes would you most want the model to distinguish correctly in a real clinical setting, and why? Discuss whether the model's performance is acceptable given the types of errors it makes.
- **A6.5:** Even though we heavily bias towards fusion it looks like it still over guessed normals as fusions and S classes. This is defenitely not a modle that sould be used and we should likely re-bias our weights so that we have less false positives. This was likely an overcorrection in biasing our weights at the start.

Prelab Questions

- **Q0.1):** Both MLPs and RNNs use the term *hidden*, but in different ways. What is the difference between a hidden layer and a hidden state, and what role does each play in its respective model?
- **A0.1):** A hidden layer in an MLP is *hidden* because the weights for the forward pass are learned and "hidden" to us as trainers. Hidden states in RNNs are similar as its hidden as the recurrent state (context and sequence history) is learned and not defined. Generally similar for the hidden part but encode different data.

- **Q0.2):** Suppose for our RNN model we set `seq_len = 1`. In what sense does an RNN become similar to a standard feed-forward neural network?
- **A0.2):** It would become similar as it wouldn't actually be recurrent and would effectively just do the standard forward pass without refeeding. (is refeeding a proper term?)

- **Q0.3):** Both RNNs and LSTMs are designed for sequential data. What additional component does an LSTM include that is not present in a vanilla RNN, and what purpose does it serve?
- **A0.3):** The LSTM has a "Cell State" which allows it to remember or forget information selectively via gating which allows the LSTM to remember or forget

over longer sequences, compared to RNNs which can struggle to learn longer sequences.